

Heterogeneous Computing

Ergebnisbericht Übung 2

In dieser Übung bin ich etwas anders vorgegangen als üblicherweise. Normalerweise versuche ich eine Programmierübung weitestgehend allein zu lösen, um das Thema und die Vorgehensweise zu verstehen. In einem zweiten Schritt bringe ich das ganze dann mit Codex in Reinform. Während dieses Prozesses kommt es natürlich häufig dazu, dass ich die KI nach einzelnen Code-Blöcken, Implementierungsdetails oder Ideen frage. Ich versuche es allerdings zu vermeiden, eine komplette Implementierung vom Agenten entgegenzunehmen.

Da ich in dieser Übung hier allerdings gar keine Ahnung hatte, wo und vor allem wie ich anfangen sollte, habe ich mich (auch aus Zeitgründen) dazu entschieden eine „Agentic“ Herangehensweise zu wählen. Ich wusste, dass ich in Python arbeiten will, da ich die Sprache gut kenne und Implementierungen problemlos lesen und verstehen kann. Aus einem früheren privaten Projekt wusste ich, dass ich das Package „Numba“ verwenden kann, um GPU-Kernel zu schreiben. Also habe ich Codex die Aufgabenstellung gegeben, angewiesen, dass Python mit Numba verwendet werden soll und eine grobe erste Struktur bauen lassen. Die Anforderung an die Architektur war, dass beide Aufgaben in einer Experiment-Suite bearbeitet werden können, um einfach und schnell Funktionalität testen zu können.

Die Implementierung beinhaltet zwei Python Skripte *compute.py* und *memory.py* für Aufgabe 1 und 2 respektive. Die restlichen Skripte sind reporting, und helper für drumherum. Das Skript *compute.py* implementiert einen uniformen sowie einen konfigurierbaren divergenten Kernel auf der GPU sowie auf der CPU. Hier werden einfache floating-point Berechnungen durchgeführt, die wenig speicherintensiv sind. Der Unterschied zwischen uniform und divergent Kernel ist, dass beim divergenten Kernel jeder der konfigurierten Degrees durchgespielt wird und entsprechend auf die einzelnen Threads verteilt wird. Das Skript *memory.py* führt speicherintensive Berechnungen durch. Auch hier gibt es unterschiedliche Kernel für die unterschiedlichen Zugriffsmuster.

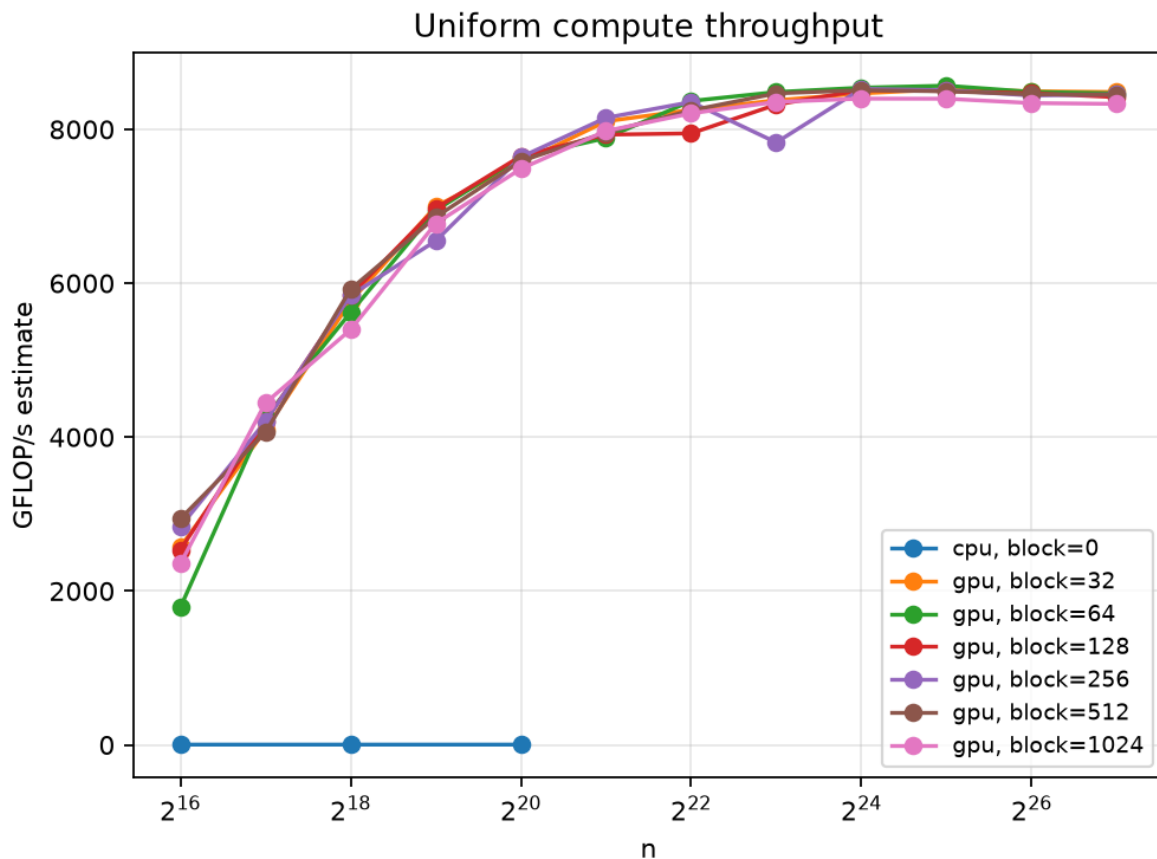
Mit Hilfe der KI konnte ich mir durch die Implementierungen relativ gut erklären, was in den Kernen passiert, was die Unterschiede sind und welche Konsequenzen die unterschiedlichen Herangehensweisen haben.

Alle Experimente wurden auf meinem Heimrechner mit Intel Core i7 9700k @ 4.7GHz, 32 GB DDR4 Ram und einer RTX 2080 Super durchgeführt. Die Parameter für die Experimente sind wie folgt:

- $N_GPU = [2^{16}, 2^{28}]$, $N_CPU = [2^{16}, 2^{18}, 2^{20}]$
- Divergence = [1, 2, 4, 8, 16, 32]
- Block Sizes = [32, 64, 128, 256, 512, 1024]

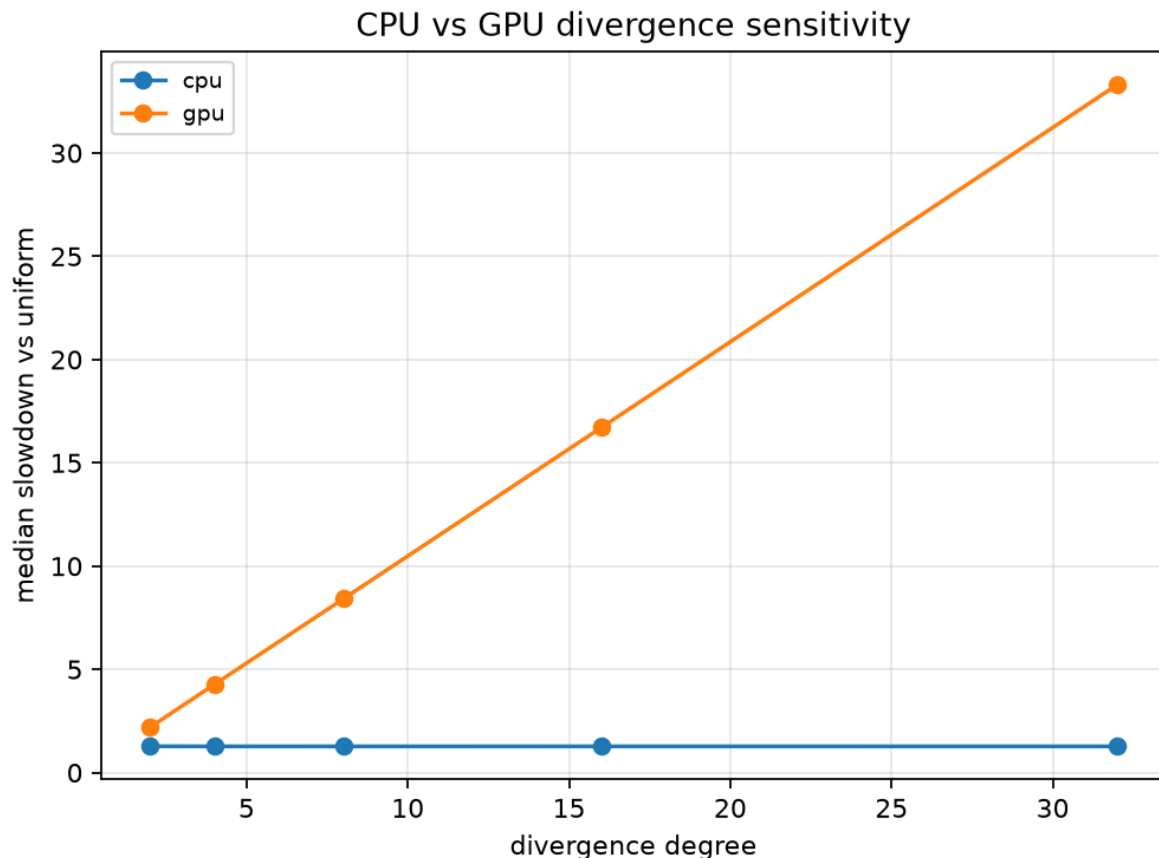
- Strides = [1, 2, 4, 8, 16, 32]
- Peak Bandwidth GPU = 495 GB/s
- Iterationen = 512
- Wiederholungen = 5

Wenn man einen Blick auf die resultierenden Plots wirft, lassen sich die Fragen aus dem Übungsblatt gut beantworten:



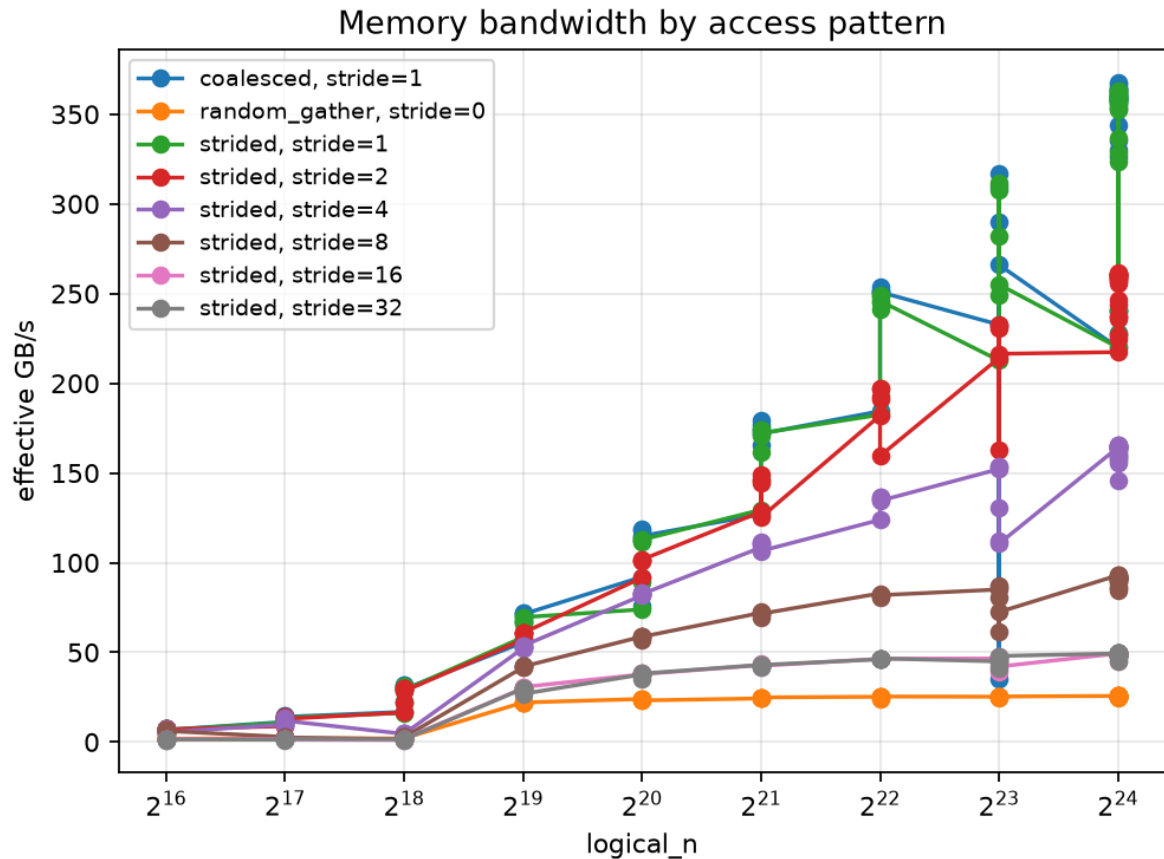
Im uniformen Fall kann man erkennen, dass die GPU ab $n=2^{22}$ etwa gesättigt ist, die Rechenleistung nimmt danach nicht mehr wesentlich zu und bei noch höheren Exponenten sogar eher ab. Meine CPU hat in diesem Szenario keine Chance mit der GPU mithalten. Die Sättigung der GPU tritt mit etwa 8 Millionen Threads ein ($n=2^{23}$). Die Blockgröße, also wie viele Warps gemeinsam gestartet werden, hat in diesem Experiment scheinbar keinen großen Einfluss auf die Rechenleistung. Das wird vermutlich daran liegen, dass die Rechenaufgabe klein, wenig speicherintensiv und unabhängig ist. Dadurch fallen Probleme wie Speicherzugriff, Warp Auslastung oder Synchronisation zwischen Blöcken weg, die mit kleinerer Blockgröße normalerweise auftreten könnten.

Wenn wir nun Divergenz einführen und Stück für Stück Threads quasi sequentiell laufen lassen, ergibt sich folgendes Bild:

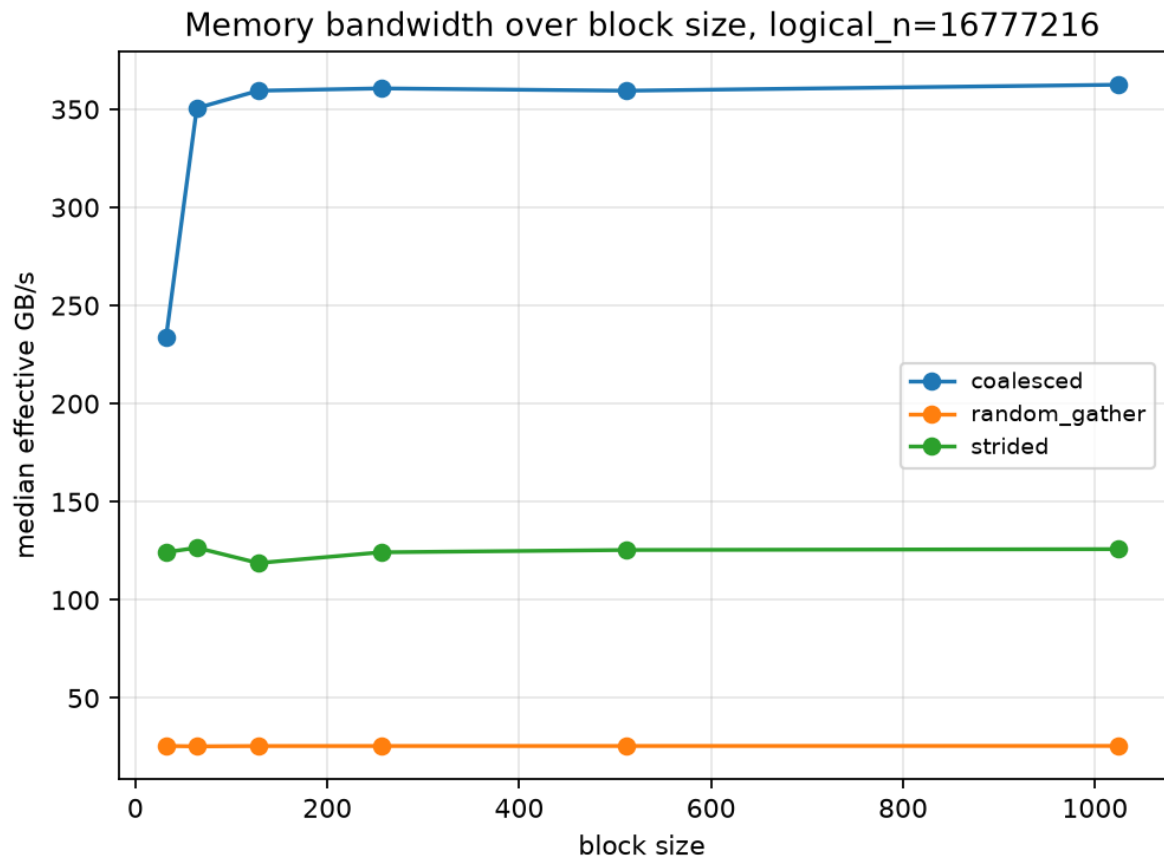


Hier wird ganz klar deutlich, dass die GPU mit steigender Divergenz deutlich langsamere Laufzeiten hat als die CPU. Das liegt in erster Linie daran, dass die GPU bei steigender Divergenz nicht mehr alle Threads synchron ausführen kann, sondern Stück für Stück Threads nacheinander ausführen muss. Im Endeffekt läuft das Programm quasi sequentiell bei maximaler Divergenz. Da die CPU kein SIMT betreibt, sondern Threads individuell parallelisiert, wirkt sich die Divergenz bei weitem nicht so stark aus wie bei der GPU. Vor allem fällt auf, dass die CPU einen nahezu konstanten Faktor der Verlangsamung bei steigender Divergenz hat.

Im Memory Szenario können wir feststellen, dass coalesced Speicherzugriff die verfügbare Bandbreite am stärksten ausnutzt. Strided Speicherzugriff ist auch in der Lage dazu die verfügbare Bandbreite gut zu nutzen allerdings nimmt dieser Effekt mit steigender Stride ab. Das liegt daran, dass bei coalesced Zugriffen die Speicherblöcke nah beieinander liegen, genauso wie die Threads, die darauf zugreifen. Da alles nah beieinander liegt benötigt es wenige Zugriffe, um die benötigten Bytes zu bekommen. Erweitert man nun den Abstand mit Stride wird der Adressraum größer. Dieselbe Anzahl an Bytes, die gelesen werden müssen, benötigen einen deutlich größeren Adressraum zu Lasten der Bandbreite. Bei einer theoretischen peak Bandbreite von 495.9 GB/s meiner RTX 2080 super sind im coalesced Fall ca. 70% Speicherauslastung drinnen, wenn wir auf $n=2^{24}$ gehen.



Die Blockgröße hingegen hat ab einer Größe von 128 keinen nennenswerten Einfluss mehr auf die Bandbreite, wie man in der folgenden Grafik erkennen kann. Die Unterschiede zwischen den Zugriffsmustern lassen sich auch hier gut erkennen. Lediglich bei einer kleinen Blockgröße, also wenigen Threads pro Block gibt es für strided und coalesced Zugriffsmuster Schwankungen. Das lässt sich dadurch erklären, dass der Scheduler nicht genügend Warps zur Verfügung hat, um die GPU vollständig zu beschäftigen. Was man dann beobachten kann, ist die echte Latenz eines Warps bei Blockgröße 32, dann passt nämlich genau ein Warp in den Block. Werden die Blöcke größer, können mehrere Warps gescheduled werden und somit versteckt sich die Wartezeit auf den Speicherzugriff von Warp 1 hinter der aktiven Rechenzeit von Warp 2 etc. Die geschätzte Occupancy ist für Blockgröße 32 50% gewesen, während größere Blöcke ab 64 eine geschätzte Occupancy von 100% hatten.



Um die GPU Speichereffizient zu nutzen, brauchen wir also Daten, die so strukturiert sind, dass sie im coalesced Fall funktionieren. Das bedeutet, dass Daten im Speicher nah beieinander liegen müssen, wenn sie von denselben Threads verwendet werden. Zum Beispiel sind einfache Arrays effizienter als Linked-Lists mit Pointern.

Zusammengefasst habe ich in dieser Übung insbesondere beim Bericht schreiben doch viel gelernt. Da ich im Bericht die Grafiken interpretieren und korrekt einordnen musste, war das Verständnis der zugrundeliegenden Theorie doch sehr wichtig. Auch wenn ich von der Implementierungsseite im ersten Schritt nur Bahnhof verstanden habe, kann ich jetzt rückblickend doch einiges davon mitnehmen. Ein grundsätzliches Problem, was ich in solchen Low-Level Programmieraufgaben oft feststelle, ist ein fehlendes Wissen und Verständnis der zugrundeliegenden Hardware und Rechnerstrukturen. Mit genügend Vorwissen über die Struktur der CPU, Caches und Pipelines, hätte ich möglicherweise besser und schneller sinnvolle Schlüsse aus der Aufgabe ziehen können. So musste ich auf die doch sehr ausführlichen Zusammenfassungen und Erklärungen der KI zurückgreifen, um die Ergebnisse der beiden Aufgabenteile korrekt zu interpretieren.